LAB REPORT – EXPERIMENT 2

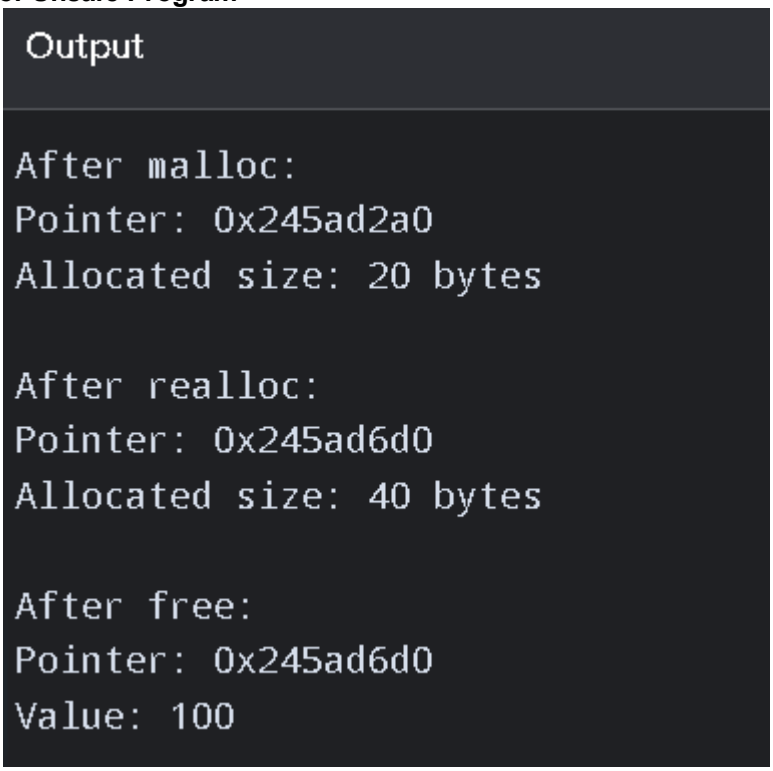
Dynamic Memory Allocation, Reallocation and Deallocation

Aim: To allocate a dynamic array using malloc(), reallocate it using realloc(), and deallocate it using free(). To demonstrate an unchecked allocation leading to NULL pointer dereference and a use-after-free through a dangling pointer, and then implement the corrected version.

1. Unsafe Program

```
#include <stdio.h>

#include <stdlib.h>


int main(void)

{

    int *ptr = malloc(5 * sizeof(int));


    printf("After malloc:\n");

    printf("Pointer: %p\n", (void *)ptr);

    printf("Allocated size: %zu bytes\n", 5 * sizeof(int));


    if (ptr == NULL)

        return 1;


    /* Reallocate */

    ptr = realloc(ptr, 10 * sizeof(int));


    printf("\nAfter realloc:\n");

    printf("Pointer: %p\n", (void *)ptr);

    printf("Allocated size: %zu bytes\n", 10 * sizeof(int));


    /* Free memory */

    free(ptr);


    printf("\nAfter free:\n");

    printf("Pointer: %p\n", (void *)ptr);


    /* ERROR: use-after-free */

    ptr[0] = 100;


    printf("Value: %d\n", ptr[0]);


    return 0;

}
```

Output of Unsafe Program



The first output shows that malloc() returned (nil) for the extremely large allocation, while the later part shows memory being allocated, reallocated, freed, and then accessed through the dangling pointer. The value read after free is invalid because the memory has already been released.

2. Security Impact of Unsafe Code

NULL pointer dereference: Using a NULL pointer as if it referred to valid memory can crash the program and may cause denial of service.

Use-after-free: Accessing memory after it has been freed is a memory-safety vulnerability that can cause crashes, data corruption, or potentially more serious security problems depending on memory reuse.

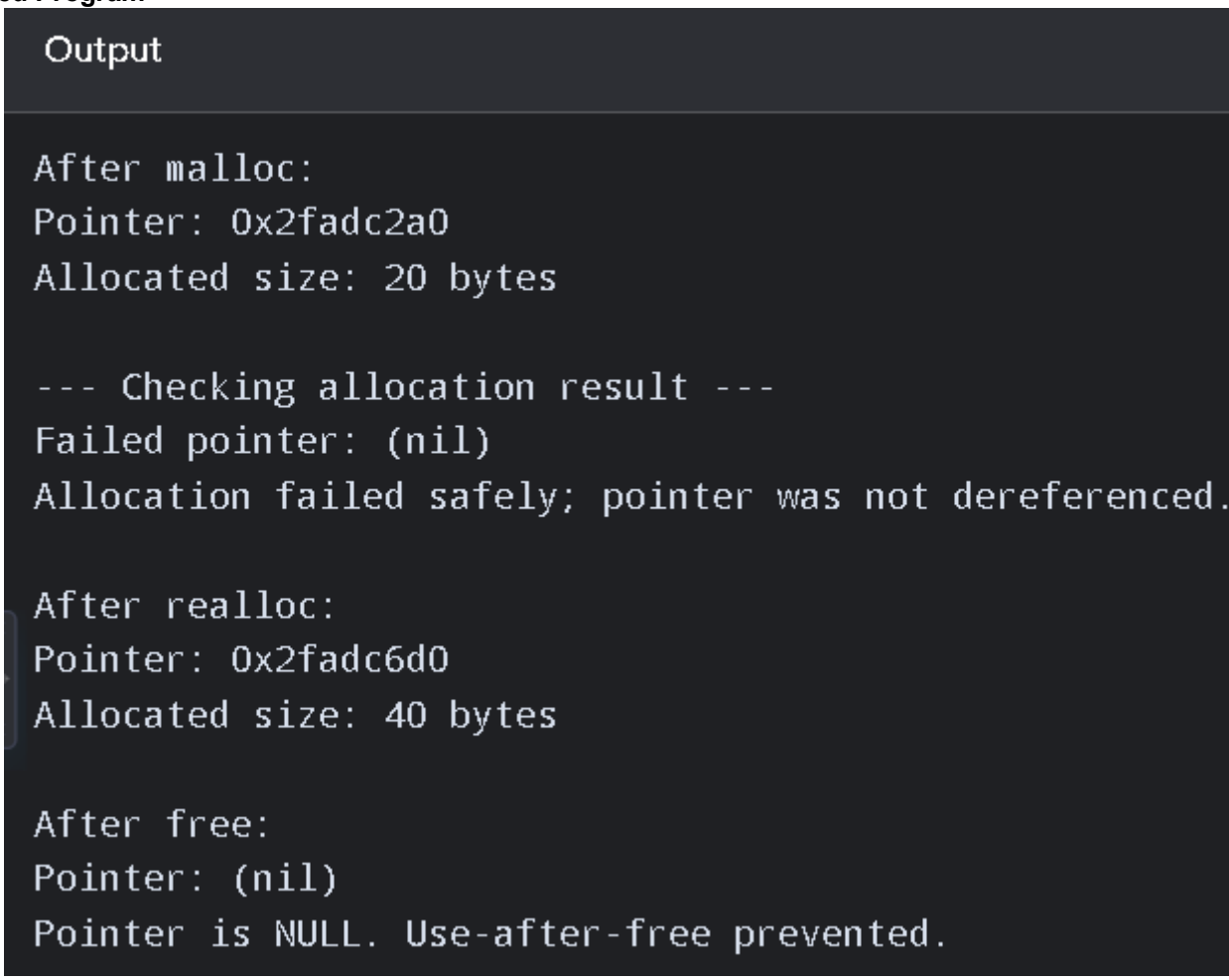
3. Corrected Program

```
#include <stdio.h>

#include <stdlib.h>


int main(void)

{

    int *ptr = NULL;

    size_t size = 5;


    /* malloc */

    ptr = malloc(size * sizeof(int));


    if (ptr == NULL)

    {

        fprintf(stderr, "malloc failed\n");

        return EXIT_FAILURE;
```

```
}

/* Initialize allocated memory */
for (size_t i = 0; i < size; i++)
{
    ptr[i] = 0;
}

printf("After malloc:\n");
printf("Pointer: %p\n", (void *)ptr);
printf("Allocated size: %zu bytes\n", size * sizeof(int));

/* Demonstrate safe handling of allocation failure */
int *failed_ptr = NULL;

printf("\n--- Checking allocation result ---\n");
printf("Failed pointer: %p\n", (void *)failed_ptr);

if (failed_ptr == NULL)
{
    printf("Allocation failed safely; pointer was not dereferenced.\n");
}

/* realloc */
size_t new_size = 10;

int *temp = realloc(ptr, new_size * sizeof(int));

if (temp == NULL)
{
    fprintf(stderr, "realloc failed\n");

    /* Original ptr is still valid */
    free(ptr);
    ptr = NULL;

    return EXIT_FAILURE;
}

ptr = temp;

/* Initialize newly allocated portion */
for (size_t i = size; i < new_size; i++)
```

```
    {
        ptr[i] = 0;
    }

    size = new_size;

    printf("\nAfter realloc:\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: %zu bytes\n", size * sizeof(int));

    /* Deallocation */
    free(ptr);

    /* Remove dangling pointer */
    ptr = NULL;

    printf("\nAfter free:\n");
    printf("Pointer: %p\n", (void *)ptr);

    /* Safe check instead of using freed memory */
    if (ptr == NULL)
    {
        printf("Pointer is NULL. Use-after-free prevented.\n");
    }

    return EXIT_SUCCESS;
}
```

Output of Corrected Program



4. Corrections Made

Problem	Correction
Unchecked malloc()	Check whether the returned pointer is NULL.
Unchecked realloc()	Use a temporary pointer and check the result before updating ptr.
Use-after-free	Do not access memory after free().
Dangling pointer	Set ptr to NULL after free().
Uninitialized new memory	Initialize the newly allocated elements after successful realloc().

5. Result

The dynamic array was successfully allocated using malloc(), expanded using realloc(), and released using free(). The unsafe version demonstrated the required NULL-pointer and use-after-free cases. The corrected version checked allocation results, safely handled realloc(), initialized the array, freed the memory, and set the pointer to NULL. The corrected output shows the pointer as (nil) after deallocation.

6. Conclusion

The experiment demonstrates the importance of checking dynamic memory allocation results and avoiding access to freed memory. Proper pointer management prevents common C memory-safety errors.